



Laboratório 2: Sockets TCP e UDP em Java

11/09/2025

i Nota

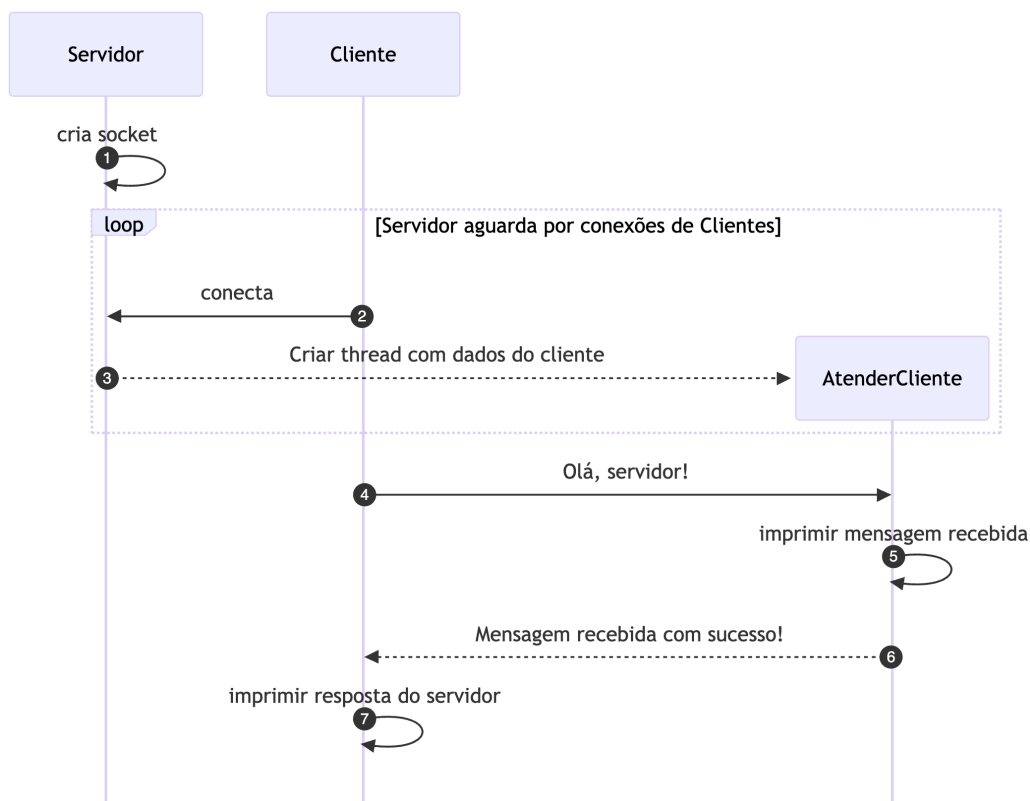
Java possui diferentes APIs para lidar com operações de I/O. Aqui serão apresentados exemplos com a API Java I/O, que segue o modelo bloqueante e é mais simples de entender. A API Java NIO.2 permite desenvolver aplicações com comunicação assíncrona, porém seu entendimento demanda mais tempo. Veja mais detalhes em <https://docs.oracle.com/en/java/javase/21/core/java-nio.html>.

1 Sockets TCP

1.1 Olá Mundo

Neste exemplo o cliente, após conectar no servidor TCP, envia uma mensagem de saudação (String) ao servidor. O servidor recebe a mensagem, a imprime no console, envia uma resposta ao cliente (String) e finaliza a conexão. O cliente aguarda o servidor responder, imprime a resposta no console e finaliza a conexão. Na Figura 1 temos o diagrama de sequência do exemplo.

Figura 1: Diagrama de sequência do exemplo Olá Mundo



Listagem 1: Servidor TCP

```
package engtelecom.std.servidor;

import java.net.ServerSocket;

public class Servidor {
    public static void main(String[] args) {
        System.out.println("\u2591\u2592\u2592 Iniciando o servidor \u2592\u2592\u2591");
        int porta = 12345;

        try (ServerSocket servidor = new ServerSocket(porta);) {

            System.out.println("Servidor aguardando conexões em " + servidor.getInetAddress() + ":" + porta);
            System.out.println("Pressione CTRL+C para encerrar o processo do servidor\n\n");

            while (true) {
                // Aceita a conexão de um cliente e cria uma nova thread para atendê-lo
                new Thread(new AtenderCliente(servidor.accept())).start();
            }
        } catch (Exception e) {
            System.err.println("Erro: " + e.getMessage());
        }
    }
}
```

Listagem 2: Thread para atender cliente

```
package engtelecom.std.servidor;

import java.io.BufferedReader;
import java.io.DataOutputStream;
import java.io.InputStreamReader;
import java.net.Socket;

public class AtenderCliente implements Runnable{

    private Socket cliente;

    public AtenderCliente(Socket cliente) {
        this.cliente = cliente;
    }

    @Override
    public void run() {
        if (cliente != null) {
            System.out.println("Cliente conectado: " + cliente.getInetAddress()+":"+cliente.getPort());
            try {
                // Estabelecendo os fluxos de entrada e saída
                // Acordo entre o cliente e o servidor, garante que serão trocadas apenas cadeias de
                caracteres codificadas em UTF-8
                BufferedReader entradaDoCliente = new BufferedReader(new InputStreamReader(cliente.
getInputStream(), "UTF-8"));
                DataOutputStream saidaParaOCliente = new DataOutputStream(cliente.getOutputStream());

                // Lendo a mensagem do cliente
                String mensagemDoCliente = entradaDoCliente.readLine();
                System.out.println("Mensagem recebida do cliente: " + mensagemDoCliente+"\n");

                // Enviando a resposta para o cliente
                saidaParaOCliente.writeBytes("Mensagem recebida com sucesso!\n");

            } catch (Exception e) {
                System.err.println("Erro: " + e.getMessage());
            }
        } else {
            System.out.println("Erro: cliente não conectado");
        }
    }
}
```

Listagem 3: Cliente TCP que envia uma String

```
package engtelecom.std.cliente;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.Socket;

public class Cliente {
    public static void main(String[] args) {
        System.out.println("\n\n\u2591\u2592\u2592 Iniciando o cliente \u2592\u2592\u2591\n");

        String enderecoServidor = "localhost";
        int porta = 12345;

        // Verifica se o usuário informou o endereço e a porta do servidor como argumentos
        if (args.length == 2) {
            enderecoServidor = args[0];
            porta = Integer.parseInt(args[1]);
        }

        // Estabelecendo a conexão com o servidor
        try (Socket cliente = new Socket(enderecoServidor, porta)) {
            System.out.println("Conectado ao servidor " + enderecoServidor + ":" + porta);

            // Estabelecendo os fluxos de entrada e saída
            BufferedReader entradaDoServidor = new BufferedReader(new InputStreamReader(cliente.
getInputStream()));
            // Somente bytes podem ser enviados e aqui estamos enviando uma string, por isso usamos um
OutputStreamWriter com UTF-8
            OutputStreamWriter saidaParaOServidor = new OutputStreamWriter(cliente.getOutputStream(), "UTF-8"
);

            // Enviando a mensagem para o servidor
            saidaParaOServidor.write("Olá, servidor!\n");
            saidaParaOServidor.flush();

            // Lendo a resposta do servidor
            String respostaDoServidor = entradaDoServidor.readLine();
            System.out.println("Resposta do servidor: " + respostaDoServidor);
        } catch (Exception e) {
            System.err.println("Erro: " + e.getMessage());
        }
    }
}
```

Pergunta

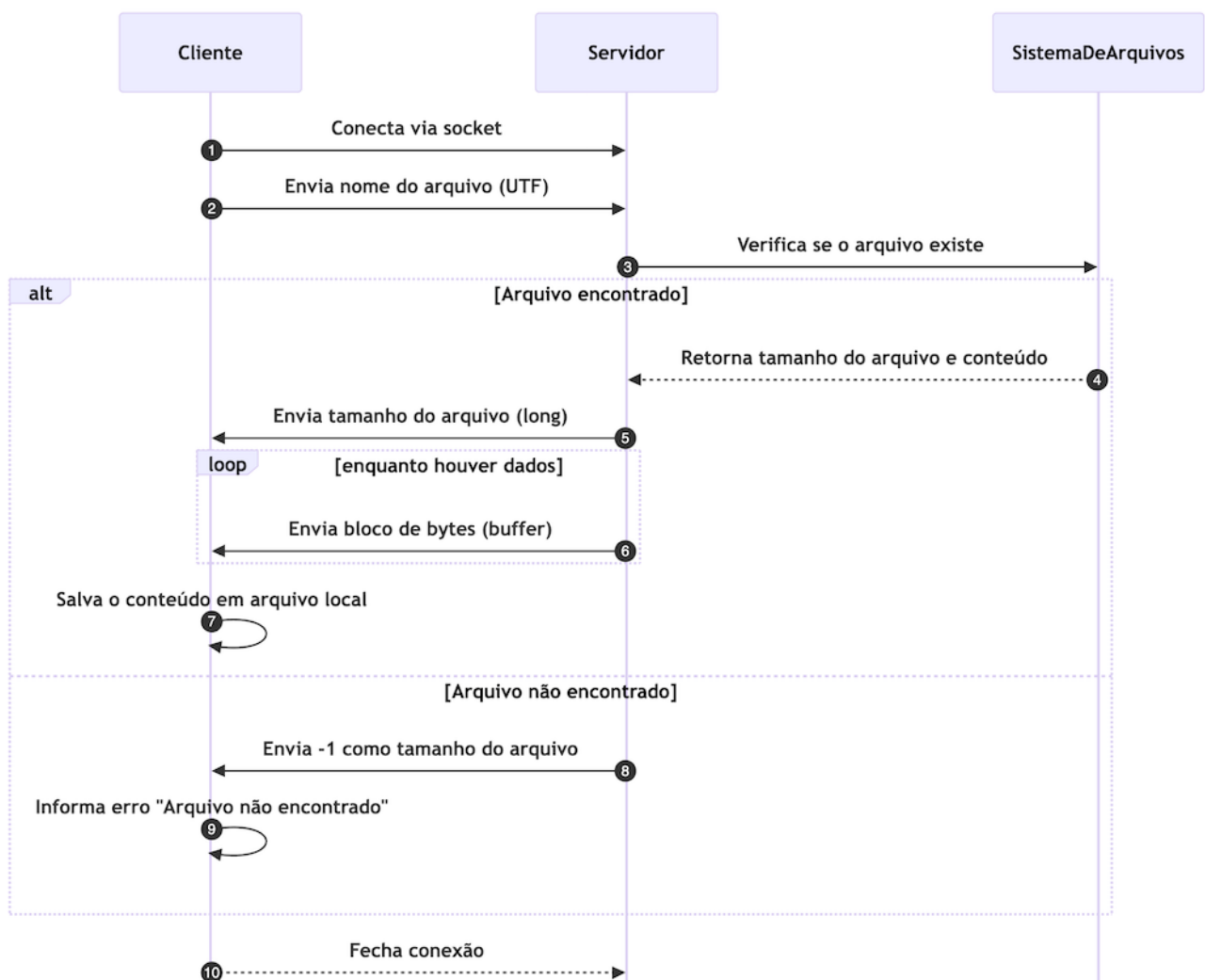
Seria possível usar o netcat para interagir com o Servidor ou com o cliente TCP desenvolvido neste laboratório? Caso seja, faça a demonstração e, se necessário, realize as modificações necessárias nos códigos. Caso não seja, explique o motivo.

1.2 Exercício: Transferência de arquivos de tamanho arbitrário

Com base no código da Subseção 1.1, faça um programa que permita transferir arquivos de tamanho arbitrário entre o cliente e o servidor. O cliente envia, ao servidor, o nome do arquivo que deseja receber. Se o arquivo existir, então o servidor envia o tamanho do arquivo e depois o arquivo (dividido em blocos de bytes de tamanho 4.096). Se não existir, então o servidor envia o valor -1 para o cliente. O cliente deve receber o tamanho do arquivo e, se o tamanho for maior que zero, deve receber o arquivo (em blocos de bytes de tamanho 4.096).

Na Figura 2 é apresentado o diagrama de sequência com as trocas de mensagens entre cliente e servidor. O servidor deverá ser *multithread*, ou seja, deve atender vários clientes simultaneamente.

Figura 2: Diagrama de sequência do exemplo transferência de arquivo



Trechos de códigos Java necessários para o desenvolvimento do exercício

Listagem 4: Trecho de código necessário para o servidor

```
// DataInputStream e DataOutputStream são usados para ler e escrever dados binários
DataInputStream dis = new DataInputStream(clientSocket.getInputStream());
DataOutputStream dos = new DataOutputStream(clientSocket.getOutputStream());

// readUTF lê uma String codificada em UTF-8
String nomeArquivo = dis.readUTF();
File arquivo = new File(nomeArquivo);

if (arquivo.exists()) {
    dos.writeLong(arquivo.length());
} else {
    dos.writeLong(-1);
}

// A leitura do arquivo deve ser feita em blocos de bytes.
byte[] buffer = new byte[4096];
int bytesLidos;

// O envio do arquivo deve ser feito em blocos de bytes até que o arquivo seja totalmente enviado.
FileInputStream fis = new FileInputStream(arquivo)
while ((bytesLidos = fis.read(buffer)) != -1) {
    // O método write do DataOutputStream envia os bytes para o cliente
    dos.write(buffer, 0, bytesLidos);
}
// É importante liberar os recursos após o uso. Talvez try-with-resources seja mais interessante
fis.close();
dos.flush();
clientSocket.close();
```

Listagem 5: Trecho de código necessário para o cliente

```
// DataInputStream
DataOutputStream dos = new DataOutputStream(socket.getOutputStream());
DataInputStream dis = new DataInputStream(socket.getInputStream());

// O cliente envia o nome do arquivo que deseja receber
dos.writeUTF(nomeArquivo);

// O cliente aguarda o servidor enviar o tamanho do arquivo. Se -1, então o arquivo não existe.
long tamanhoArquivo = dis.readLong();

// O arquivo será salvo com o mesmo nome que o arquivo enviado pelo servidor
FileOutputStream fos = new FileOutputStream(nomeArquivo)

byte[] buffer = new byte[4096];
long total = 0;
int bytesLidos;

// O cliente lê o arquivo em blocos de bytes e escreve no arquivo local
// Math.min é necessário para evitar que a leitura do buffer ultrapasse o tamanho do arquivo
while (total < tamanhoArquivo && (bytesLidos = dis.read(buffer, 0, (int)Math.min(buffer.length,
    tamanhoArquivo - total))) != -1) {
    // O método write do FileOutputStream escreve os bytes no arquivo local
    fos.write(buffer, 0, bytesLidos);
    // Atualiza o total de bytes lidos
    total += bytesLidos;
}

// É importante liberar os recursos após o uso. Talvez try-with-resources seja mais interessante
fos.close();
dis.close();
socket.close();
```

2 Sockets UDP

Listagem 6: Servidor UDP

```
package engtelecom.std.udp;

import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;

public class ServidorUdp {

    private int porta;
    private int bufferSize;

    public ServidorUdp(int porta) {
        this.porta = porta;
        this.bufferSize = 1024;
    }

    private void aguardarMensagem(DatagramSocket socket, byte[] buffer) throws IOException {
        // Cria um pacote para receber os dados
        DatagramPacket pacoteRecebido = new DatagramPacket(buffer, buffer.length);

        // Aguarda a chegada de um pacote
        System.out.println("Aguardando pacote...");
        socket.receive(pacoteRecebido);

        // Exibe os dados recebidos
        String mensagem = new String(pacoteRecebido.getData(), 0, pacoteRecebido.getLength());
        System.out.println("Mensagem recebida: " + mensagem);

        // Envia a resposta
        String r = "Olá, eu sou o servidor UDP!";
        byte[] resposta = r.getBytes();
        DatagramPacket pacoteResposta = new DatagramPacket(resposta, resposta.length, pacoteRecebido.
            getAddress(), pacoteRecebido.getPort());
        socket.send(pacoteResposta);
    }

    public void iniciarServidor() {
        try (DatagramSocket socket = new DatagramSocket(this.porta)) {
            // Cria um buffer de bytes para receber os dados
            byte[] buffer = new byte[bufferSize];

            System.out.println("\u2591\u2592\u2592 Servidor UDP iniciado na porta " + this.porta);
            System.out.println("Pressione CTRL+C para encerrar...\n");

            while(true) {
                aguardarMensagem(socket, buffer);
            }
        } catch (Exception e) {
            System.err.println("Erro: " + e.getMessage());
        }
    }

    public static void main(String[] args) {

        // Cria um servidor UDP na porta 9876
        ServidorUdp servidor = new ServidorUdp(9876);

        // Inicia o servidor
        servidor.iniciarServidor();
    }
}
```

Listagem 7: Cliente UDP

```
package engtelecom.std.udp;
```

```

import java.net.DatagramPacket;
import java.net.DatagramSocket;

public class ClienteUdp {

    private String host;
    private int porta;

    public ClienteUdp(String host, int porta) {
        this.host = host;
        this.porta = porta;
    }

    public void comunicacao(String mensagem) {
        try (DatagramSocket socket = new DatagramSocket()) {
            // Converte a mensagem para bytes
            byte[] buffer = mensagem.getBytes();

            // Cria um pacote com os dados da mensagem e o endereço do servidor
            DatagramPacket pacote = new DatagramPacket(buffer, buffer.length, java.net.InetAddress.getByName(
                this.host), this.porta);

            // Envia o pacote
            socket.send(pacote);
            System.out.println("Mensagem enviada: " + mensagem);

            // Cria um pacote para receber os dados
            DatagramPacket pacoteRecebido = new DatagramPacket(buffer, buffer.length);

            // Aguarda a chegada de um pacote
            System.out.println("Aguardando resposta...");
            socket.receive(pacoteRecebido);

            // Exibe os dados recebidos
            String resposta = new String(pacoteRecebido.getData(), 0, pacoteRecebido.getLength());
            System.out.println("Resposta do servidor: " + resposta);
        } catch (Exception e) {
            System.err.println("Erro: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        System.out.println("\u2591\u2592\u2592 Cliente UDP \u2592\u2592\u2591");

        String host = "localhost";
        int porta = 9876;

        // Verifica se o usuário informou o endereço e a porta do servidor como argumentos
        if (args.length == 2) {
            host = args[0];
            porta = Integer.parseInt(args[1]);
        }

        // Cria um cliente UDP
        ClienteUdp cliente = new ClienteUdp(host, porta);

        // Envia uma mensagem para o servidor
        cliente.comunicacao("Olá, eu sou o cliente UDP!");
    }
}

```

3 Multicast

Acesse <https://github.com/emersonmello/sockets-java> e execute o exemplo de *multicast* com o servidor e cliente em contêineres Docker diferentes. Mas atenção, é necessário que ambos estejam na mesma rede. Não há ordem específica para execução do servidor e cliente.

CC BY Documento licenciado sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/).